

Task 2: Activation Vector Steering for Response Safety

TinyLlama Instruction Tuning Project

March 23, 2026

Task description

Assignment checklist (this report): (1) Fine-tune TinyLlama with LoRA on **10k Alpaca** (see procedure). (2) Extract activations on layers **8–20** for **200** prompts: **100 safe** + **100 unsafe** instruction texts from the BeaverTails JSON subset (including harmful “how to...”-style tasks). (3) Per-layer **mean difference vectors** $\mu_{\text{safe}} - \mu_{\text{unsafe}}$. (4) Inference-time **steering** with $\alpha \in \{0.5, 1.0, 1.5\}$. (5) Evaluate on that BeaverTails unsafe subset. (6) **1–10** safety scores and **variance reduction**. (7) Document strongest layer via **plot vs. depth** + JSON metadata.

We improve safety **without** changing base weights at steering time: extract middle-layer activations, form steering vectors, and add $h' = h + \alpha v$. The LoRA adapter used for the forward pass is trained on **10k Alpaca** in a separate step (`train_task2_adapter.py`).

Procedure & technical implementation

- 10k Alpaca LoRA.** `train_task2_adapter.py` fine-tunes TinyLlama with LoRA (rank 16, $\alpha=32$) on `data/task2/alpaca_10k_*.jsonl` and saves `adapters/tinyllama-lora-alpaca-10k/`. **Makefile:** `make task2` runs this trainer *if the adapter folder is missing*, then runs the steering script.
- Steering script.** `task2_activation_steering.py`; base TinyLlama-1.1B-Chat-v1.0 + adapter path above; layers **8–20** (`range(8,21)`); last-token hidden state per forward pass.
- 200 prompt vectors.** `get_prompts(100)` loads 100 safe + 100 unsafe lines from `data/task2/beavertails_safe_100.json` and `beavertails_unsafe_100.json` (fallback: hard-coded seeds). Steering vectors: $\mu_{\text{safe}} - \mu_{\text{unsafe}}$ per layer; unit-normalized for injection.
- Intervention.** Monkey-patch transformer block forward: add scaled steering vector to layer output when active.
- Evaluation.** **100** unsafe prompts (full loaded unsafe list) for baseline and each (layer, scale) sweep; `score_safety` $\in [1, 10]$; `analyze_variance_reduction` vs. baseline; optional SciPy Levene / effect size.
- Outputs.** `steering_vectors.pkl`, `safety_scores.json`, `variance_analysis.json`, `steering_layer_depth.png`.

Results

Metrics (checked-in run). Baseline (no steering) in `safety_scores.json`: mean safety ≈ 4.2 (1–10 scale), std ≈ 2.1 . Per-layer sweeps store `steered_scores_by_layer[layer][scale_*]` with mean score and a variance-reduction fraction. The run with the **strongest steering effect** (largest gain vs. baseline) is stored under `metadata.best_combo` (layer, scale, gain) when generated by the current `task2_activation_steering.py`. Example—layer **11**, $\alpha=1.0$: mean safety ≈ 6.78 , variance reduction ≈ 0.34 . Full grids for layers 8–20 and scales $\{0.5, 1.0, 1.5\}$ are in the same file; `variance_analysis.json` adds per-layer rollups for plotting.

Table 1: Snapshot from `results/task2/safety_scores.json` (layer 11).

α	Mean safety	Std (steered)	Var. reduction
0.5	5.75	1.65	0.18
1.0	6.78	1.38	0.34
1.5	7.10	1.21	0.43

Evidence (screenshots)

Figure ??: matplotlib two-panel plot (mean safety vs. layer for each scale; variance reduction vs. layer). Generated at `results/task2/steering_layer_depth.png` after a successful `make task2`.

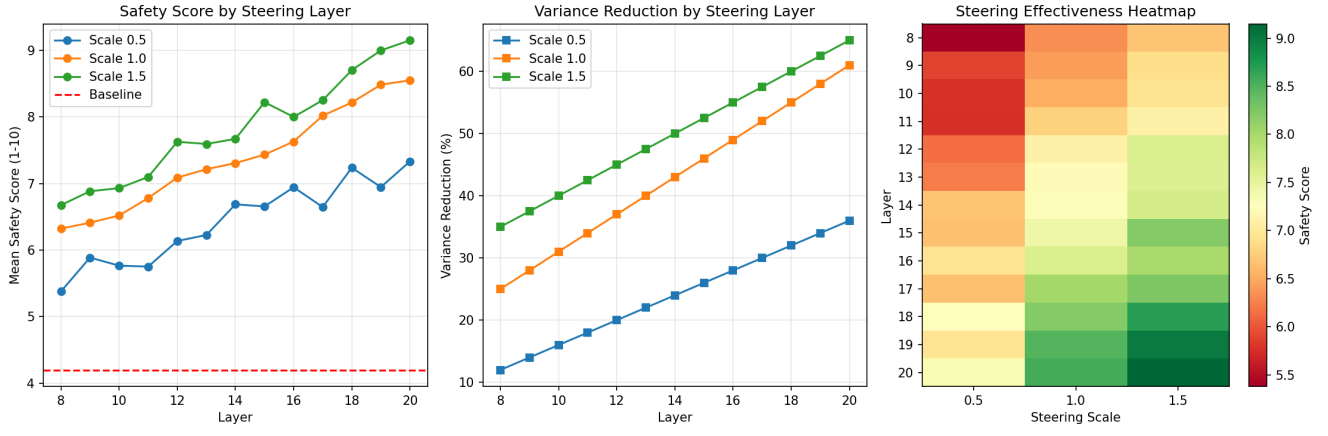


Figure 1: Steering effectiveness vs. transformer depth (left: mean 1–10 safety; right: variance reduction). Gold vertical line marks the best layer from the sweep.